

Szoftverfejlesztési folyamat és elemei; a folyamat különböző modelljei

TÖRTÉNETI HÁTTÉR: A SZOFTVERKRÍZIS

- 1960-as évek végén, ahogy a hardverek teljesítménye nőtt, úgy a szoftverek komplexitása is robbanásszerűen emelkedett
 - ennek köszönhetően a projektek rendszeresen túllépték a határidőket és a költség keretet is, illetve a termékek tele voltak hibákkal és nehezen is voltak karbantarthatóak, ez az állapot a szoftverkrízis

Szoftverfejlesztési folyamat

- ez volt a szoftverkrízisre a válasz
- **szoftverfejlesztési folyamatnak** nevezzük egy bizonyos modell szerinti tevékenységek összességét, ezek a tevékenységek a szoftvertermék létrehozására, kezelésére és karbantartására fókuszálnak
- a szoftverfejlesztési folyamat **minőségi szoftver biztosítéka**
 - **folyamat láthatósága**
 - a menedzsment is láthatja, hol tart a projekt, a mérföldkövek és a dokumentumok által
 - **költséghatékonyság**
 - minél korábban fedezünk fel egy hibát a folyamat során, annál olcsóbb kijavítani
 - pl: tervezésnél még olcsó kijavítani, de miután már kész és akkor derül ki a hiba akkor már drága
- **Fázisai**
 1. **specifikáció** (mit csináljunk)
 - a rendszer funkcióinak és a működési megkorlátozásoknak (pl: válaszidő, biztonság) a meghatározása
 - lépései: megvalósíthatósági tanulmány, követelmények feltárása és elemzése, dokumentálás, validáció
 - kritikus szakasz, mert itt a legolcsóbb a javítás
 2. **fejlesztés** (tervezés és implementáció - hogyan csináljuk)
 - **tervezés:** architektúra (alrendszerek), interfészek (hogyan kommunikálnak az elemek), adatstruktúrák és algoritmusok kidolgozása
 - **implementáció:** kódolás és nyomkövetés (debugging, hibák megtalálása és kijavítása)
 3. **validáció és verifikáció**
 - **Verifikáció**
 - "jól építjük a rendszert?" - megfelel-e a specifikációnak
 - **Validáció**
 - "a megfelelő terméket építjük?" - megfelel-e a felhasználói, megrendelői igénynek

■ tesztelési szintek:

- egységteszt / unit teszt (programozó végzi)
 - a komponens tesztelése többi komponenstől függetlenül
- modulteszt (integrációs teszt)
 - függő és kapcsolódó komponensek tesztelése együtt
- alrendszereszt
 - több modul közös illeszkedésének vizsgálata
 - általában független csapat által, aki nem rajtuk dolgozott
- rendszerteszt
 - alrendszereket közösen teszteljük
- átvételi teszt (alfa: fejlesztőnél, béta: felhasználónál)

4. evolúció (változás kezelése)

■ egy szoftver soha nincs kész

- a környezet változik, pl: új operációs rendszer
- új igények jelennek meg a megrendelőnél vagy a felhasználónál

■ pl: egy apphoz valami törvényi szabályozás miatt új funkciót kell adni

● **Fő elemek**

- **feladatok:** konkrét munkatevékenység (pl: xy funkció lefejlesztése)
- **termékek:** a tevékenységek eredményei (pl: forráskód, dokumentáció, tesztjegyzőkönyv)
 - nem minden terméket kap meg a megrendelő, pl: belső kód felülvizsgálati jegyzőkönyvet nem kapja meg
- **határidők:** időpontok amikre bizonyos eredményeknek el kell készülniük
- **átadandók:** olyan termékek melyeket a megrendelőnek hivatalosan átadunk

● **Kiegészítő elemek (esernyőtevékenységek)**

- **projektmenedzsment:** ütemezés és erőforráskezelés
- **konfigurációkezelés:** verziókövetés
- **minőségbiztosítás:** standardok betartása és ennek ellenőrzése
- **kockázatmenedzsment:** potenciális veszélyek (pl: kulcsemberek felmondása) azonosítása és kezelése
- **mérés:** folyamat és a termék jellemzőinek számszerűsítése (pl. hány hiba jut 1000 sor kódra)

● **Eszközök**

- automatizált támogatást segítik
- pl: IDE-k, verziókezelők, CASE eszköz és 4GT
- **CASE eszköz**
 - magát a szoftverfejlesztési folyamat automatizálását támogatja
 - **funkciói:** kódgenerálás diagramokból, automatikus dokumentáció készítés és tesztadat generálás
 - **Upper-CASE**
 - folyamat elejét támogatja
 - követelményelemzés, tervezés és modellezés
 - pl: UML diagram tervezők

- **Lower-CASE**
 - folyamat végét támogatja
 - kódolás, tesztelés, hibakeresés, verziókezelés
- **4GT (negyedik generációs technikák)**
 - lehetővé teszi a fejlesztőnek, hogy magas szinten határozza meg **mit** kell csinálni és az eszköz automatikusan generálja le a **hogyan-t** (a kódot)
 - veszélye, hogy a generált kód nem mindig hatékony és nehéz karbantartani

SZOFTVERFOLYAMAT MODELLEK

- szoftverfolyamat modellje egy absztrakt leírása a fejlesztés menetének
- nincs 1 tökéletes modell, hanem a projekt jellegétől és méretétől, illetve a követelmények stabilitásától is függ, melyiket éri meg választani

Vízesés modell (Waterfall model)

- legrégebbi, klasszikus modell
- a tevékenységeket szigorú sorrendbe, egymás után kell végezni
 - tehát egy szekvenciális modell, ahol a fázisok lépcsőzetesen követik egymást

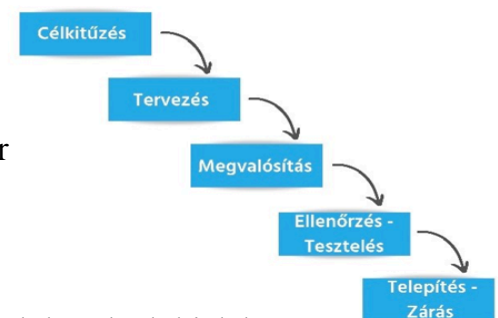
● Fázisai

1. követelményelemzés és specifikáció
 - vevővel közösen rögzítésre kerül a rendszer funkciói és korlátai
 - eredménye: követelményspecifikáció
2. rendszer- és szoftvertervezés
 - az architektúra, az adatstruktúra és a felületek kerülnek kialakításra
3. implementáció (kódolás) és egységtesztelés (unit teszt)
 - kód megírása és az egységek külön-külön tesztelése
4. integráció és rendszertesztelés
 - az egységek összeépítésre kerülnek (modul) és a specifikáció alapján a teljes rendszer ellenőrzésre kerül, illetve a modulok is (integrációs teszt)
5. üzemeltetés és karbantartás
 - szoftver éles használata, illetve fellépő hibák javítása és kisebb módosítások

● Jellemzői

- merev struktúra
 - egy fázisnak be kell fejeződnie teljesen és a kimeneteit (dokumentumait) jóvá kell hagyni, mielőtt a következő fázis kezdődne
 - minden fázis végén van review, felülvizsgált (kontroll pontok)

Vízesés modell



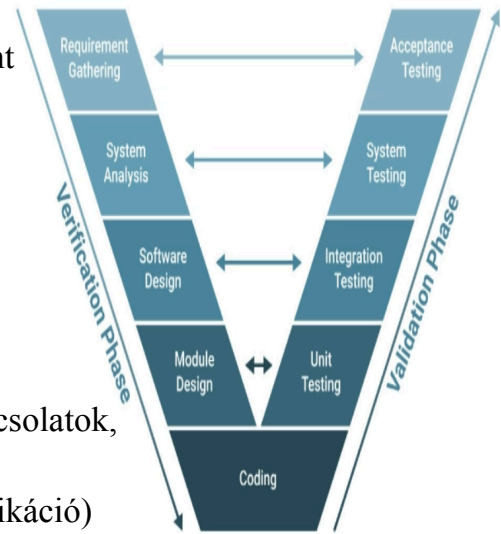
- dokumentum-vezérelt
 - minden fázis végén rengeteg dokumentációt kell készíteni
- a visszacsatolás, elméletileg lehetséges, de gyakorlatban visszalépni az előző fázisba nehéz és drága
- **előnyei**
 - jól strukturált, könnyen átlátható a menedzsment számára
 - világos mérföldkövek és fegyelmezett munkamenet
- **hátrányai**
 - **rugalmatlanság:** menet közben szinte lehetetlen a követelményeken változtatni
 - **a blokkolt állapot problémája:** a fejlesztők egymásra várnak
 - fő gond, hogy **a működő szoftver csak a folyamat legvégén jelenik meg** és eddig a megrendelő nem találkozik vele
 - ezáltal a **hibák halmozódnak**, mivel minden fázison visszük tovább
 - ha ekkor derül ki egy tervezési hiba, vagy valami félreértés volt a megrendelő igényeiről akkor a javítás költsége hatalmas lehet
- **mikor válasszuk?**
 - csak akkor ajánlott a vizesés modell használata, ha a **követelmények az elején 100%-ban tiszták és rögzítettek**, azaz a specifikáció nagyon jó és a projekt során várhatóan nem fognak változni
 - ez így ritkán igaz, mivel az elején egy projektet nehéz ennyire jól átlátni
 - pl: egy meglévő rendszer átírása más programozási nyelvre, mert ilyenkor a funkcionalitások adottak és nem változnak
 - ha a rendszer **kritikus biztonsági elemeket** tartalmaz, ahol a precíz dokumentáció és a többszöri ellenőrzés fontosabb, mint a sebesség
 - pl: repülésirányítás vagy orvosi szoftverek

V-modell

- a vízesés modell kiterjesztése, amely a minőségbiztosításra helyezi a hangsúlyt
- a "V" betű bal szára a fejlesztési/tervezési fázisokat, a jobb szára a tesztelési fázisokat jelöli

- **Szintek**

- minden tervezési szinthez tartozik egy tesztelési szint
- 1. specifikáció ↔ átvételi teszt (validáció)
 - azt teszteljük, hogy a legelső fázisban rögzített üzleti igények teljesültek-e
- 2. rendszer tervezés ↔ rendszerteszt
 - a teljes architektúra és a funkcionális követelmények ellenőrzése
- 3. alrendszer tervezés ↔ integrációs teszt
 - az egyes modulok, komponensek közötti kapcsolatok, interfészek tesztelése
- 4. komponens tervezés ↔ egységteszt, unit teszt (verifikáció)
 - a legkisebb kód egységek (függvények, osztályok) tesztelése
- 5. kódolás



- **Működési sajátosságok**

- **korai teszttervezés**
 - nem várjuk meg a kódolás végét a tesztek kitalálásával, hanem amint kész a specifikáció, egyből megírjuk hozzá az átvételi tesztek forgatókönyvét, stb
 - pl: a webshopban "legyen kosár funkció", rögtön mellé írjuk azt is: "a teszt akkor sikeres, ha 3 termék behelyezése után a végösszeg pontos"
 - ez a fő különbsége a vízesés modellhez képest
- **szigorú minőségbiztosítás**: minden szinten van kontrollpont
- **előnye**, hogy sokkal hamarabb kiderülnek a hibák, mint a sima vízesés modell esetén, mert a tesztelők már a specifikáció fázisában bekapcsolódnak
 - így jobb minőségű végterméket eredményez, mint a sima vízesés
- **problémája**, hogy ugyanolyan rugalmatlan mint a vízesés modell, tehát menet közben nehézkes a változtatás, mert ha új igény érkezik, akkor a "V" egész bal oldalát és a hozzátartozó tesztterveket újra kell írni a jobb oldalon
- **mikor válasszuk?**
 - ha kell a magas megbízhatóság, azaz ahol a hibák kritikusak
 - pl: orvosi műszerek szoftverei, banki tranzakciós rendszerek
 - pl: egy gyógyszerértári nyilvántartó rendszernél, ahol minden módosítást szigorúan dokumentálni és tesztelni kell törvényi előírások miatt

ITERATÍV ÉS EVOLÚCIÓS MODELLEK

- az **iteratív modellek** esetén nem egyetlen nagy lépésben akarjuk a végterméket elő állítani hanem több ciklusban (iterációban) finomítjuk a szoftvert még kész nem lesz
- az **evolúciós modelleknél** a szoftver folyamatosan fejlődik a visszajelzések alapján

Prototípus (evolúciós) modell

- alapötlete, hogy mielőtt belekezdenénk egy drága, teljes körű fejlesztésbe, először készítsünk inkább egy prototípust, vázlatot, amin a felhasználó láthatja a működést és az alapvető funkciókat
- **folyamat lépései**
 1. követelmények gyűjtése
 - csak a legfontosabb célok meghatározására
 2. gyors tervezés
 - csak a látható részekre fókuszál (pl: felületek)
 3. prototípus megépítése
 - gyorsan összerakott, de nem feltétlenül optimális kód
 4. kiértékelés a vevővel
 - félreértések itt derülnek ki
 5. finomítás / újrakezdés
 - visszajelzések alapján javítjuk a prototípust, extrém esetben újrakezdjük vagy elkezdhetjük a valódi fejlesztést
- **két fő típusa van a prototípusnak**
 - **eldobható**
 - csak követelmények tisztázására kell a prototípus, utána kidobjuk és tiszta lappal indul a valódi fejlesztés
 - **evolúciós**
 - a prototípus lesz az alapja a végterméknek, azaz ez kerül folyamatosan bővítésre addig amíg kész nem lesz
- **mikor válasszuk?**
 - amikor a megrendelő nem tudja pontosan megfogalmazni az igényeit
 - bonyolult felhasználói interfészek (GUI) tervezésekor
 - pl: egy új mobilappnál először csak kattintható képernyőterveket mutatunk a befektetőknek, hogy lássák a logikát, mielőtt megírnánk a szerveroldali kódot

Spirál modell

- ez egy kockázatvezérelt modell, mely ötvözi az iteratív természetet a vízésés modell kontrollált szempontjaival
 - egyetlen modell ahol a kockázat kezelés minden iterációban jelen van, nem csak szoftvert építünk hanem folyamatosan vizsgáljuk, hogy érdemes-e folytatni
- nem fix fázisokból áll, hanem ciklusokból (spirál), ahol minden kör egyre többbe kerül, de egyre közelebb visz a kész szoftverhez
 - illetve nincs fix körszám, akkor ér véget mikor elkészül a szoftver, vagy ha a kockázatelemzés során kiderül, hogy a projektet inkább le kell állítani

- a folyamat spirálisan halad kifelé négy szektoron keresztül:

1. célkitűzés

- mit akarunk elérni ebben a körben?
- korlátok és alternatívák meghatározása

2. kockázatelemzés

- mi romolhat el?
 - pl: nem tetszik a vevőnek, lassú a rendszer, stb
- legfontosabb egyedi jellemzője ennek a modellnek
- prototípusokat készítünk a bizonytalanságok csökkentésére

3. fejlesztés és tesztelés

- a szoftver egy részének tényleges elkészítése és tesztelése

4. review (visszatekintés) és tervezés, majd következő körre (iterációra) átlépés

- a következő kör előkészítése, eredmények áttekintése



- **működési sajátosság**

- a legfontosabb elem a kockázat, ha a kockázat túl nagy (pl: túl drága technológia), a projektet bármelyik kör végén leállíthatjuk

- **mikor használjuk?**

- nagy, bonyolult és kockázatos rendszereknél ajánlott és ideális
- olyan projekteknél ahol a technológia újszerű és bizonytalan
- pl: katonai radarrendszer, ahol minden fázisban tesztelni kell, hogy a technológia képes-e arra ami nekünk kell, még mielőtt milliárdokat költenénk

WINWIN spirális modell

- a spirális modell kiterjesztése amely a **tárgyalásra fókuszál**
- lényege, hogy a fejlesztés elején a résztvevők (stakeholder-ek) gyakran ellentétes érdekekkel rendelkeznek
 - pl: a megrendelő olcsón akar sokat, a fejlesztő profitot akar és reális határidőt
- **működése**
 - mindenki meghatározza a saját “win-feltételét”, amivel ő boldog lenne
 - megkeresik az ütközéseket
 - tárgyalás útján olyan kompromisszumot kötnek, amivel mindenki nyer (win-win)
- **mikor válasszuk?**
 - sok szereplős projekt esetén, ahol a megrendelői oldalon több különböző érdekcsoport van
 - pl: egyetemi Neptun rendszer, ahol vannak oktatók akik egyszerűbb adminisztrációt akarnak, míg a hallgató szép mobilfelületet, a fenntartó meg alacsony szerver költségeket, a WINWIN modell segít egyensúlyt találni ezek között

INKREMENTÁLIS ÉS GYORS FEJLESZTÉSI MODELLEK

- ezen megközelítések a vízésés modell (szigorúság) és az evolúciós modellek (rugalmasság) előnyeit ötvözi

Inkrementális modell

- ez is a vízésés modell variációja, ahol a szoftvert nem egyszerre adjuk át, hanem darabokban (inkrementumokban) szállítjuk
 - minden inkremens a szoftver egy-egy funkcionális részét valósítja meg
- **működése**
 - a fejlesztés elején meghatározásra kerül az architektúra és a főbb funkciók
 - a funkciókhoz prioritásokat rendelünk
 - a legmagasabb prioritású funkciót tartalmazó inkremens készül el legelőbb
 - minden újabb inkremens ami később jön hozzáadódik a már meglévőkhöz
- **előnyök**
 - a felhasználó hamarabb kap működő szoftvert, így a visszacsatolás gyorsabb, ezáltal előbb kiszűrhető az, ha a specifikáció nem pontos, vagy a megrendelő igényeinek nem felel meg
 - a legfontosbb modulok fejlesztődnek le először, így ezek lesznek a legteszteltebbek is
 - ha ki is futunk az időből, a főbb funkciók már készen lehetnek, bukásra kevesebb az esély
- **hátrányok**
 - nehéz meghatározni az inkremensek optimális méretét
 - ha túl kicsi nem működőképes
 - ha túl nagy elveszik az előnye
 - vannak alap funkciók amik nélkül semmi nem működik, így ezek megvalósítása elnyújtja az első szállítást
- **mikor használjuk?**
 - ha a piacra kerülés sebessége fontos és a rendszer jól felbontható független funkciókra
 - pl: egy szövegszerkesztő fejlesztésekor az 1. inkremens tudjon csak gépelni és menteni, a 2. már formázni, a 3. pedig helyesírást ellenőrizni

RAD (Rapid Application Development)

- egy nagysebességű szoftverfejlesztési folyamat, amely az inkrementális modell párhuzamosítására és komponens alapú építkezésre épít
 - rendkívül rövid fejlesztési idő: **60-90 nap**
 - erős **komponens-újrafelhasználás** és automatizált kódgenerálás (4GT eszközök)
- **fázisa**
 1. üzleti modellezés
 - milyen információk kellenek a funkciókhoz
 2. adatmodellezés
 - adatszerkezetek finomítása
 3. adatfolyam processzus
 - adatkezelés megvalósítása
 4. alkalmazás generálás
 - automatikus kódgenerálás, kész elemek beillesztése
 5. tesztelés
 - főleg komponensek illeszkedésére fókuszál
- **problémái**
 - hatalmas emberi erőforrás igény
 - sok csapat kell a párhuzamos munkához
 - csak jól modularizálható rendszereknél alkalmazható
 - ha nem bontható jól modulokra, akkor a párhuzamos csapatok egymásra fognak várni és idő veszik el
- **mikor használjuk?**
 - nagyon szoros határidő esetén, ha van elég pénz és ember a párhuzamos fejlesztésre
 - pl: egy cégnek hirtelen szüksége van egy belső nyilvántartóra egy új jogszabály miatt, és 2 hónapja van rá

SPECIÁLIS MODERN MEGKÖZELÍTÉSEK

Újrafelhasználás-orientált (komponens alapú) fejlesztés

- nem a programozásra, hanem az integrációra, beépítésre helyezi a hangsúlyt
 - tehát már létező, tesztelt szoftverkomponenseket építünk össze egy rendszerre
- **folyamat lépései**
 1. komponens analízis
 - megnézzük, milyen kész elemek érhetők el a piacon, vagy a belső könyvtárunkba
 2. követelmények módosítása
 - ha a kész komponens nem pontosan azt tudja amit a megrendelő el vár, akkor gyakran a vevőnek kell alkalmazkodnia és érdekesebb követelményt módosítania, mint egy új komponens kifejlesztése
 3. rendszertervezés újrafelhasználással
 - az architektúrát a meglévő elemek köré építjük
 4. fejlesztés és integráció
 - a maradék, részek megírása és az elemek összekapcsolása
- **mikor használjuk?**
 - webes szolgáltatásoknál (pl: fizetési lehetőség integrálása)
 - ha a fejlesztési idő kritikusabb, mint az egyedi funkcionalitás
 - pl: egy webshop fejlesztésekor nem írunk saját bankkártya-kezelőt, hanem bekötjük a PayPal vagy a Stripe kész modulját

Formális módszerek (tisztaszoba módszer)

- matematikai alap, a szoftvert nem teszteljük, hanem matematikai úton bebizonyítjuk, hogy helyesen működik
 - formális specifikációt készítünk, majd kóddá transzformáljuk ezt
- **probléma:** nagyon drága és speciális matematikai tudást igényel és a megrendelő sem érti a specifikációt
- **mikor használjuk?**
 - kritikus rendszereknél, ahol az emberi élet vagy hatalmas pénz összeg a tét (pl: atomreaktor vezérlés, űrkutatás, vasúti jelzőrendszer)
 - pl: egy repülőgép robotpilóta szoftverénél nem elég a tesztelés, matematikailag garantálni kell, hogy bizonyos bemenetekre soha nem ad hibás választ

AGILIS SZOFTVERFEJLESZTÉS

- extrém iteratív modell
- agilis fejlesztés nem egy konkrét módszer, hanem egy **szemléletmód**
- alapvetésüket az **Agile Manifesto** rögzíti:
 - fontosabb a **működő szoftver**, mint az átfogó dokumentációt
 - fontosabb a **válaszkészség a változásra**, mint a terv követését
 - fontosabbak az **egyének és interakciók**, mint a folyamatok és eszközök
 - fontosabb az **együttműködés az ügyféllel**, mint a szerződéses tárgyalás
- a manifesztóban szereplő dolgok nem azt jelentik, hogy nincs terv vagy dokumentáció, hanem, hogy ha választani kell akkor a működő szoftver és a megrendelő elégedettsége elsőbbséget élvez

eXtreme Programming (XP)

- agilis módszertanok egyik előfutára
- **lényege**
 - nagyon rövid (akár néhány napos) ciklusok és apró inkrementek
 - **folyamatos megrendelői jelenlét**
 - **páros programozás**, két programozó ül egy gép előtt, az egyik ír a másik ellenőrzi
 - ez azért hatékony, mert a kód-felülvizsgálat (review) azonnal, valós időben történik, ezért kevesebb a hiba, és a tudás megoszlik a csapattagok között
- gyakran káosznak tűnhet, sok dokumentációt elhagy és nehezen skálázható óriási csapatokra
- **mikor használjuk?**
 - kisméretű, rugalmas csapatoknál, ahol a megrendelői követelmények folyamatosan és radikálisan változnak
 - pl: egy innovatív startup új algoritmusának fejlesztése, ahol naponta jönnek új ötletek a befektetőktől

Scrum

- egy agilis **keretrendszer**, csak váz és nem kész recept
- **Szerepkörök**
 - **Product Owner (PO)**
 - a megrendelőt és a menedzsmentet (fejlesztői cég) képviseli, ő menedzseli a feladatlistát (product backlog), ő dönt a prioritásokról is
 - **Scrum Master**
 - a folyamat őre, segíti a csapatot, elhárítja az akadályokat, védi a csapatot a külső zavaró tényezőktől
 - nem projektmenedzser, azaz nem főnök
 - **Fejlesztői csapat**
 - önszerveződő, kereszt-funkcionális csapat (fejlesztők, tesztelők, UX-esek együtt alkotják)

- **Események**

- **Sprint (futam)**

- a fejlesztés rövid, fix hosszúságú (általában 1-4 hetes) ciklusokban, azaz sprintekben zajlik
 - a cél egy sprinten belül nem változhat

- **Sprint Planning (tervezés)**

- kiválasztásra kerülnek a product backlogról, hogy mit csinál a csapat az adott sprintben

- **Daily Scrum (stand-up)**

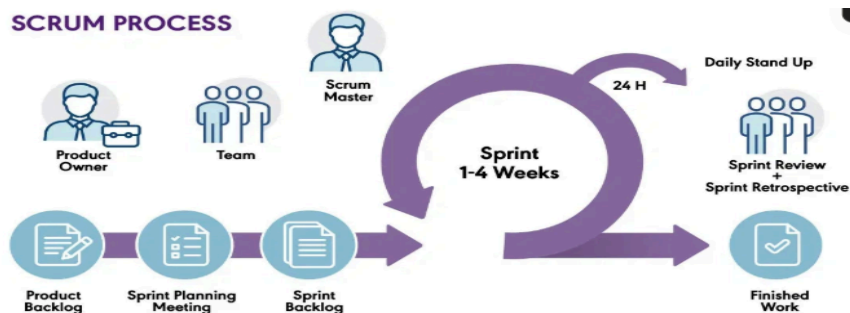
- napi max 15 perces megbeszélés amit a scrum master vezet, a fejlesztők közül mindenki elmondja, hogy:
 - mit csinált a legutóbbi daily scrum óta
 - volt-e valami akadályozó tényező eközben
 - mit fog csinálni a mai napon

- **Sprint Review (demó)**

- a sprint végén a működő szoftver bemutatásra kerül a PO-nak és a megrendelőknek

- **Sprint Retrospective (visszatekintés)**

- a csapat belső folyamatainak értékelése és javítása, mindenki elmondja mit változtatna, mit csinálna másképp és, hogy mi maradjon



- **Termékek**

- **Product Backlog**

- a teljes követelménylista prioritási sorrendben

- **Sprint Backlog**

- az adott sprintre kiválasztott feladatok listája

- **Inkremens**

- sprint végére elkészült ("Kész") állapotú, működő szoftverrész
 - a kész állapot nem csak annyi, hogy a kód meg van, hanem a csapatnak van egy előre rögzített listája (Definition of Done), aminek minden pontja teljesülnie kell: pl. tesztelve van, dokumentálva van, átment a kódátnézésen

- **mikor használjuk?**

- ha gyorsan változó piaci igények és bizonytalan követelmények vannak melyek változhatnak

Kanban

- legfontosabb eleme egy tábla, melyben oszlopok (pl: teendő (TODO), folyamatban lévő (IN PROGRESS) és kész (DONE), de tetszőlegesen bármi igazából) vannak amikbe a tennivalókat, feladatokat rendezzük
- folyamat vizualizációjára és a folyamatban lévő munkák korlátozására való (pl: ha in progressbe van akkor más már ne kezdjen bele, stb)
- fejlesztés során a tábla folyamatosan frissül, ezzel reprezentálható a fejlesztés aktuális státusza
- SCRUM tök jól bővíthető vele
- pl: JIRA, Git issue tracker, stb

